



endingo

정훈이의 공부일지

CATEGORY



👋 안녕하세요, AI 개발자 이정훈입니다

AI · 백엔드 · 데이터 분석 중심의 프로젝트와 경험을 공유합니다.

 Contact Me

Data science/딥러닝

[딥러닝] 6. 다중 클래스 분류 모델링 - Mnist

endingo 2025. 4. 7. 20:28

1. 데이터 다운로드

1) 데이터 다운로드

Download training data from open datasets.

```
train_dataset = datasets.MNIST(  
    root="data",  
    train=True,  
    download=True,  
    transform=ToTensor(), # 픽셀값을 [0,1] 사이로 정규화하고 텐서로 변환  
)
```

Download test data from open datasets.

```
test_dataset = datasets.MNIST(  
    root="data",  
    train=False,  
    download=True,  
    transform=ToTensor(),  
)
```



정훈이의 공부일지



train_dataset, test_dataset

```
(Dataset MNIST
  Number of datapoints: 60000
  Root location: data
  Split: Train
  StandardTransform
  Transform: ToTensor(),
Dataset MNIST
  Number of datapoints: 10000
  Root location: data
  Split: Test
  StandardTransform
  Transform: ToTensor())
```

- 데이터셋의 x와 y

train_dataset.data.shape, train_dataset.targets.shape

```
(torch.Size([60000, 28, 28]), torch.Size([60000]))
```

train_dataset.data.shape: 60000건의 28x28 데이터

train_dataset.targets.shape: 60000건의 타겟

- y의 클래스 구성

train_dataset.classes

```
['0 - zero',
 '1 - one',
 '2 - two',
 '3 - three',
 '4 - four',
 '5 - five',
 '6 - six',
 '7 - seven',
 '8 - eight',
 '9 - nine']
```

- 데이터 1건에 대한 정보 확인하기

train_dataset[0]

- 데이터 이미지 확인

plt.imshow(image, cmap='gray')

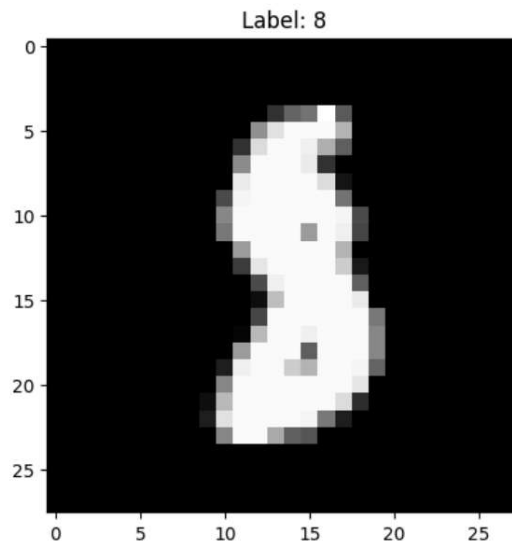
n = 3432



정훈이의 공부일지



```
plt.imshow(image, cmap=gray)
plt.title(f"Label: {label}")
plt.show()
```



2) 데이터 준비

DataLoader: 어레이 => 텐서 => 텐서 정보 결합 => 데이터로더로 딥러닝에 전달

```
batch_size = 64
train_dataloader = DataLoader(train_dataset, batch_size=batch_size)
```

- 잘 전달되었나 확인해보기드

1) 데이터 다운로드

Download training data from open datasets.

```
train_dataset = datasets.MNIST(
    root="data",
    train=True,
    download=True,
    transform=ToTensor(), # 픽셀값을 [0,1] 사이로 정규화하고 텐서로 변환
)
```

Download test data from open datasets.

```
test_dataset = datasets.MNIST(
    root="data",
    train=False,
    download=True,
```



정훈이의 공부일지



- 데이터셋 일반 정보

```
train_dataset, test_dataset
```

```
[Obj]
```

- 데이터셋의 x와 y

```
train_dataset.data.shape, train_dataset.targets.shape
```

```
[Obj]
```

`train_dataset.data.shape`: 60000건의 28x28 데이터

`train_dataset.targets.shape`: 60000건의 타겟

- y의 클래스 구성

```
train_dataset.classes
```

```
[Obj]
```

- 데이터 1건에 대한 정보 확인하기

```
train_dataset[0]
```

- 데이터 이미지 확인

```
plt.imshow(image, cmap='gray')
```

```
n = 3432
```

```
image, label = train_dataset.data[n], train_dataset.targets[n]
```

```
# 이미지 시각화
```

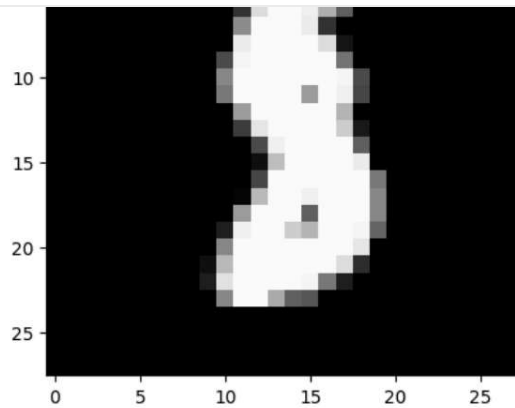
```
plt.imshow(image, cmap="gray")
```

```
plt.title(f"Label: {label}")
```

```
plt.show()
```



정훈이의 공부일지



2. 데이터 준비

1) train 데이터 로더

DataLoader: 어레이 => 텐서=> 텐서 정보 결합 => 데이터로더로 딥러닝에 전달

```
batch_size = 64
```

```
train_dataloader = DataLoader(train_dataset, batch_size=batch_size)
```

- 잘 전달되었나 확인해보기

첫번째 배치만 로딩해서 살펴보기

```
for X, y in train_dataloader:
```

```
    print(f"Shape of X [batch, channels, height, width]: {X.shape}")
```

```
    print(f"Shape of y: {y.shape} {y.dtype}")
```

```
    break
```

```
Shape of X [batch, channels, height, width]: torch.Size([64, 1, 28, 28])
Shape of y: torch.Size([64]) torch.int64
```

x:

- **64**: 미니 배치 사이즈 (한 번에 64개의 데이터를 모델에 넣는다는 뜻)
- **1**: 채널 수 (흑백 이미지는 채널이 1개, 컬러는 3개)
- **28, 28**: 이미지의 가로 x 세로 크기 (MNIST 같은 경우 28x28)

y:

- **torch.Size([64])**: 64개의 라벨이 있다는 뜻 (X의 각 이미지에 대한 정답)
- **torch.int64**: 정수형 자료형으로, 각 레이블이 0~9 사이 숫자 (예: MNIST의 숫자 분류 문제)



정훈이의 공부일지



```
y_val, y_test = test_dataset.targets[:5000], test_dataset.targets[5000:]
```

```
(torch.Size([5000, 28, 28]), torch.Size([5000]))
```

5000 건의 28x28 5000건의 정답레이

3) 스케일링(이미지니까 255로 화면이 구성되어있어서 255로 많이 나누는 형태로 함)

```
x_val = x_val / 255
x_test = x_test / 255
```

4) 3차원 데이터 셋을 4차원 데이터셋으로

numpy의 reshape: 같은 형태로 만들어서 모델이 이해하기 쉽게 만들어
view

```
x_val = x_val.view(5000, 1, 28, 28)
x_test = x_test.view(5000, 1, 28, 28)
```

```
print(x_val.shape, x_test.shape)
```

3. 모델링

1) 모델선언 784 개의 특성이 10개의 특성으로 출력

nn.Flatten: 이미지를 옆으로 펼치기
활성화함수는 목적함수에 포함

```
n_feature = 28 * 28
n_class = 10
```

모델 구조 설계

```
model = nn.Sequential(nn.Flatten(),                # 이미지를 옆으로 펼치기(한 행에 데이터를 넣기)
                      nn.Linear(28*28, n_class)
                      ).to(device)
```

```
print(model)
```

2) 목적함수, 옵티마이저

목적함수: nn.CrossEntropyLoss()



정훈이의 공부일지



```
optimizer = Adam(model.parameters(), lr=0.001)
```

3) 모델 학습

```
train()
```

```
evaluate()
```

```
epochs = 5
```

```
tr_loss_list, val_loss_list = [], []
```

```
for t in range(epochs):
```

```
    tr_loss = train(train_dataloader, model, loss_fn, optimizer, device)
```

```
    val_loss, _ = evaluate(x_val, y_val, model, loss_fn, device)
```

```
    # 리스트에 loss 추가 --> learning curve 그리기 위해.
```

```
    tr_loss_list.append(tr_loss)
```

```
    val_loss_list.append(val_loss)
```

```
    print(f"Epoch {t+1}, train loss : {tr_loss:4f}, val loss : {val_loss:4f}")
```

- 학습 곡선으로 잘 학습됐나 확인

초기 웨이트값이 랜덤이라 성능이 다 다를 수 있다.

```
dl_learning_curve(tr_loss_list, val_loss_list)
```

4) 모델평가

```
_, pred = evaluate(x_test, y_test, model, loss_fn, device)
```

```
pred[:5]
```

```
tensor([[ -1.2062,  1.0545,  0.3104, -0.5838,  0.2086,  2.3702, -0.4354,  1.2303,
         -2.3212, -0.5900],
        [ -1.2165,  1.0250,  0.3084, -0.6045,  0.2406,  2.3431, -0.4096,  1.2629,
        -2.2975, -0.5538],
        [ -1.2238,  1.0204,  0.2776, -0.6336,  0.2495,  2.3204, -0.4299,  1.2726,
        -2.3026, -0.5264],
        [ -1.2254,  1.0367,  0.2985, -0.6151,  0.1921,  2.3507, -0.4407,  1.2188,
        -2.2692, -0.5655],
        [ -1.2051,  1.0011,  0.2896, -0.6471,  0.2629,  2.3406, -0.3892,  1.2461,
        -2.3175, -0.5718]])
```

- 예측 결과가 나오면 softmax로 확률화

```
pred = nn.functional.softmax(pred, dim=1)
```



정훈이의 공부일지



```
tensor([[2.1366e-03, 1.4415e-03, 7.7042e-04, 8.4462e-01, 7.0237e-04, 3.4642e-01,
        1.5809e-05, 2.1009e-05, 4.4005e-03, 8.4232e-04],
        [6.0715e-04, 3.7922e-07, 4.1125e-03, 7.3671e-03, 3.7069e-01, 4.8115e-03,
        9.3708e-04, 3.3117e-03, 2.4844e-01, 3.5972e-01],
        [3.0330e-06, 2.2399e-10, 4.5384e-07, 3.2373e-06, 6.2364e-03, 3.1528e-06,
        1.2565e-06, 9.6546e-04, 3.2074e-03, 9.8958e-01],
        [8.5131e-07, 6.8029e-09, 2.0117e-06, 6.7209e-06, 9.3935e-07, 5.0752e-04,
        1.0583e-07, 1.2972e-07, 9.9896e-01, 5.2222e-04],
        [7.6182e-05, 9.1130e-11, 3.9605e-06, 1.9445e-07, 9.9788e-01, 2.6572e-05,
        7.0780e-05, 7.9076e-05, 1.7967e-04, 1.6794e-03]])
```

- 확률화했으면 가장 확률이 높은 것들을 선택

```
pred = np.argmax(pred.cpu().numpy(), axis = 1)
pred
```

5) 혼동행렬 시각화, 수치화

```
cm = confusion_matrix(y_test.numpy(), pred)
cm
```

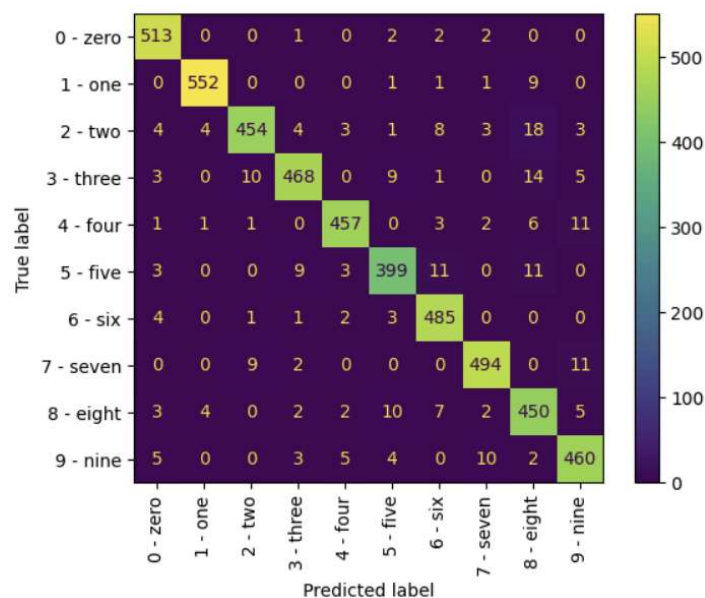
```
print(classification_report(y_test.numpy(), pred))
```

6) 혼동행렬 매트릭스로 시각화

```
ConfusionMatrixDisplay()
```

```
# confusion matrix 시각화
```

```
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels = train_dataset.classes)
disp.plot()
plt.xticks(rotation=90)
plt.show()
```





정훈이의 공부일지



마지막 활성화함수는 추가 안해도됨 => 목적함수에 softmax를 품고있어서
이런식으로 신경망의 출력값을 2의 n제곱 형태로 많이 하지만 정해진건아니다.

국룰이다.

```
n_feature = 28 * 28
n_class = 10
```

모델 구조 설계

```
model = nn.Sequential(nn.Flatten(),          # 이미지를 옆으로 펼치기(한 행에 데이터를 넣기)
                      nn.Linear(n_feature, 128),
                      nn.ReLU(),
                      nn.Linear(128, 64),
                      nn.ReLU(),
                      nn.Linear(64, n_class),
                      ).to(device)

print(model)
```

```
Sequential(
  (0): Flatten(start_dim=1, end_dim=-1)
  (1): Linear(in_features=784, out_features=128, bias=True)
  (2): ReLU()
  (3): Linear(in_features=128, out_features=64, bias=True)
  (4): ReLU()
  (5): Linear(in_features=64, out_features=10, bias=True)
)
```

공감

구독하기

[Data science](#) > [딥러닝](#) 카테고리의 다른 글

[딥러닝] 5. 다중 클래스 분류 모델링 - Iris (0)

[딥러닝] 4. 이진분류 모델링 (0)

[딥러닝] 3. 회귀 모델링 (0)

[딥러닝] 2. Pytorch (0)

[딥러닝] 1. 딥러닝 개요 (0)



정훈이의 공부일지



정훈이의 공부일지

개발하면서 겪는 시행착오와 배움을 기록합니다. 백엔드, AI, 데이터 분석, 프로젝트 경험 중심으로 운영 중입니다. 공부하면서 얻은 인사이트를 함께 나누고 싶어요!

구독하기

댓글 0

이름

비밀번호

로그인 댓글만 허용한 블로그입니다

등록

공지사항

최근에 올라온 글

- [딥러닝] 6. 다중 클래스 분류 모...
- [딥러닝] 5. 다중 클래스 분류 모...
- [딥러닝] 4. 이진분류 모델링
- [딥러닝] 3. 회귀 모델링

최근에 달린 댓글

Total

188

Today	9
Yesterday	114

링크

TAG

more

- git
- github

« 2025/04 »

일	월	화	수	목	금	토
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30			

글 보관함

- 2025/04 (16)
- 2025/03 (7)

Blog is powered by Tistory / Designed by Tistory



이정훈 개발자

AI와 백엔드, 데이터 분석을 좋아하는 개발자입니다.
다양한 문제 해결과 창의적인 개발을 추구합니다.

이메일 보내기